TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE010/DATA STRUCTURE AND ALGORITHM	**SECTION:**	CPE21S2
DATE PERFORMED:	02/10/26	**DATE SUBMITTED:**	02/10/26
NAME:	Bungag, Marc Jhowell M.	**INSTRUCTOR:**	Engr. Jimlord Quejado

HOA 10.1 Graphs

1. PROCEDURE OUTPUT

What procedures did you perform?

ILOA

CODE	OUTPUT

```cpp
#include <iostream>
// stores adjacency list items
struct adjNode {
int val, cost;
adjNode* next;
};

// structure to store edges
struct graphEdge {
int start_ver, end_ver, weight;
};

class DiaGraph{
    // insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
    adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head; // point new node to current head
        return newNode;
}

int N; // number of nodes in the graph
public:
    adjNode **head; //adjacency list as array of pointers
    // Constructor
    DiaGraph(graphEdge edges[], int n, int N) {
    // allocate new node
    head = new adjNode*[N]();
    this->N = N;
    // initialize head pointer for all vertices
    for (int i = 0; i < N; ++i)
        head[i] = nullptr;
        // construct directed graph by adding edges to it
            for (unsigned i = 0; i < n; i++) {
                int start_ver = edges[i].start_ver;
                int end_ver = edges[i].end_ver;
                int weight = edges[i].weight;
        // insert in the beginning
        adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);
        // point head pointer to new node
        head[start_ver] = newNode;
        }
}
```

```
A.1 Create a Graph
Vertices: A, B, C, D, E, F

A.2 Adjacency Matrix

Adjacency Matrix

  A B C D E F
A 0 2 4 0 0 0
B 0 0 0 0 3 0
C 0 0 0 2 0 0
D 0 4 0 0 0 0
E 0 0 0 3 0 0
F 0 0 0 0 0 0

A.3 Adjacency List
(Start Vertex, End Vertex, Weight)
(A, C, 4)
(A, B, 2)
(B, E, 3)
(C, D, 2)
(D, B, 4)
(E, D, 3)
```

```
// Destructor
~DiaGraph() {
for (int i = 0; i < N; i++)
    delete[] head[i];
    delete[] head;
}
};

// print all adjacent vertices of given vertex
void display_AdjList(adjNode* ptr, int i) {
    while (ptr != nullptr) {
    std::cout << "(" << i << ", " << ptr->val << ", " << ptr->cost << ") ";
    ptr = ptr->next;
}

std::cout << std::endl;
}

// graph implementation
int main() {
// graph edges array.
graphEdge edges[] = {
// (x, y, w) -> edge from x to y with weight w
{0,1,2},{0,2,4},{1,4,3},{2,3,2},{3,1,4},{4,3,3}
};

int N = 6; // Number of vertices in the graph
// calculate number of edges
int n = sizeof(edges)/sizeof(edges[0]);
// construct graph
DiaGraph diagraph(edges, n, N);
// print adjacency list representation of graph
std::cout<<"Graph adjacency list "<<std::endl<<"(start_vertex, end_vertex, weight):"<<std::endl;

for (int i = 0; i < N; i++) {
    // display adjacent vertices of vertex i
    display_AdjList(diagraph.head[i], i);
}
return 0;
}
```

ILOB.1

DFS

CODE

```
DFS.h > depth_first_search<T>(const Graph<T>&, size_t)
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

#define DFS_H
#ifdef DFS_H

template <typename T>
class Graph;

template <typename T>
struct Edge {
    size_t src;
    size_t dest;
    T weight;
    // To compare edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge<T> &e) const {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G) {
    for (auto i = 1; i < G.vertices(); i++) {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
        os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}
```

OUTPUT

```
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
PS C:\Users\TIPQC\Documents\HOA 10p1>
```

```cpp
template <typename T>
class Graph
{
public:
    // Initialize the graph with N vertices
    Graph(size_t N) : V(N) {
    }
    // Return number of vertices in the graph
    auto vertices() const {
        return V;
    }
    // Return all edges in the graph
    auto &edges() const {
        return edge_list;
    }
    void add_edge(Edge<T> &&e) {
        // Check if the source and destination vertices are within range
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }

    // Returns all outgoing edges from vertex v
    auto outgoing_edges(size_t v) const {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
                edges_from_v.emplace_back(e);
        }
        return edges_from_v;
}
```

```cpp
// Overloads the << operator so a graph be written directly to a stream
// Can be used as std::cout << obj << std::endl;
template <typename U>
friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);
private:
    size_t V; // Stores number of vertices in graph
    std::vector<Edge<T>> edge_list;
};

template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest) {
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    stack.push(1); // Assume that DFS always starts from vertex ID 1
    while (!stack.empty()) {
        auto current_vertex = stack.top();
        stack.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end()){
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex)){
                // If the vertex hasn't been visited, insert it in the stack.
                if(visited.find(e.dest) == visited.end()) {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}
```

```cpp
template <typename T>
auto create_reference_graph(){
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};
    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});
    return G;
}

template <typename T>
void test_DFS() {
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;
    // Run DFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order)
    std::cout << v << std::endl;
}

#endif
```

```cpp
#include <iostream>
#include "DFS.h"

int main() {
    using T = unsigned;
    test_DFS<T>();
    return 0;
}
```

ILOB.2

BFS	
CODE	OUTPUT

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>
template <typename T>
class Graph;

#define BFS_H
#ifdef BFS_H

template <typename T>
struct Edge{
    size_t src;
    size_t dest;
    T weight;
    inline bool operator<(const Edge<T> &e) const{
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const{
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G){
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}
```

```cpp
template <typename T>
class Graph{
    public:
        Graph(size_t N) : V(N) {}
        auto vertices() const{
            return V;
        }
        auto &edges() const{
            return edge_list;
        }
        void add_edge(Edge<T> &&e){
            if (e.src >= 1 && e.src <= V &&
                e.dest >= 1 && e.dest <= V)
                edge_list.emplace_back(e);
            else
                std::cerr << "Vertex out of bounds" << std::endl;
        }
        auto outgoing_edges(size_t v) const{
            std::vector<Edge<T>> edges_from_v;
            for (auto &e : edge_list)
            {
                if (e.src == v)
                {
                    edges_from_v.emplace_back(e);
                }
            }
            return edges_from_v;
        }
        template <typename U>
        friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);
private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};
```

```
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},

BFS Order of vertices:
1
2
5
4
8
3
6
7
```

```cpp
template <typename T>
auto create_reference_graph(){
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};
    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});
    return G;
}

template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest){
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    queue.push(1); // Assume that BFS always starts from vertex ID 1
    while (!queue.empty()){
        auto current_vertex = queue.front();
        queue.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end()){
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
                queue.push(e.dest);
        }
    }
    return visit_order;
}
```

```cpp

template <typename T>
void test_BFS(){
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;
    // Run BFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order)
        std::cout << v << std::endl;
}

#endif
```

```cpp
#include <iostream>
#include "BFS.h"

int main(){
    using T = unsigned;
    test_BFS<T>();
    return 0;
}
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task.

The algorithm most helpful for accomplishing this task is Depth-First Search (DFS). DFS starts from one vertex and explores as far as possible along one path before backtracking to the previous vertex. After backtracking, it explores other unvisited vertices connected to the same vertex. This process continues until all reachable vertices have been visited, which closely matches the situation described.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

The equivalent of DFS in tree traversal strategies is depth-first tree traversal, which includes Preorder, Inorder, and Postorder traversal. These traversals explore the nodes deeply along branches before moving to other branches. In Preorder, the root is visited before its children; in Inorder, the left subtree is visited before the root and then the right subtree; and in Postorder, the children are visited before the root. Like DFS, these tree traversals systematically explore one branch before moving to another.

,

3. In the performed code, what data structure is used to implement the Breadth First Search?

The data structure used to implement Breadth-First Search (BFS) is a queue. A queue follows the First-In, First-Out (FIFO) principle, meaning that the vertex that was added first is processed first. The queue allows BFS to visit vertices level by level, starting with the vertices closest to the starting vertex before moving to vertices farther away.

4. How many times can a node be visited in the BFS?

In BFS, a node can be visited only once. A visited array or another similar data structure is normally used to keep track of vertices that have already been visited. This prevents BFS from repeatedly processing the same node, especially when the graph contains cycles or multiple edges leading to the same vertex.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

COMPUTER ENGINEERING

QUEZON CITY

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.